



CS & IT ENGINEERING



Algorithms

Sorting Algorithms

Lecture No.- 03



By- Aditya sir

Recap of Previous Lecture



Topic

Topic

Bubble
Selection

Topics to be Covered



Topic

Topic

Topic

Insertion

Radix

PYQs



About Aditya Jain sir

1. Appeared for GATE during BTech and secured AIR 60 in GATE in very first attempt - City topper
2. Represented college as the first Google DSC Ambassador.
3. The only student from the batch to secure an internship at Amazon. (9+ CGPA)
4. Had offer from IIT Bombay and IISc Bangalore to join the Masters program
5. Joined IIT Bombay for my 2 year Masters program, specialization in Data Science
6. Published multiple research papers in well known conferences along with the team
7. Received the prestigious excellence in Research award from IIT Bombay for my Masters thesis
8. Completed my Masters with an overall GPA of 9.36/10
9. Joined Dream11 as a Data Scientist
10. Have mentored 12,000+ students & working professions in field of Data Science and Analytics
11. Have been mentoring & teaching GATE aspirants to secure a great rank in limited time
12. Have got around 27.5K followers on Linkedin where I share my insights and guide students and professionals.

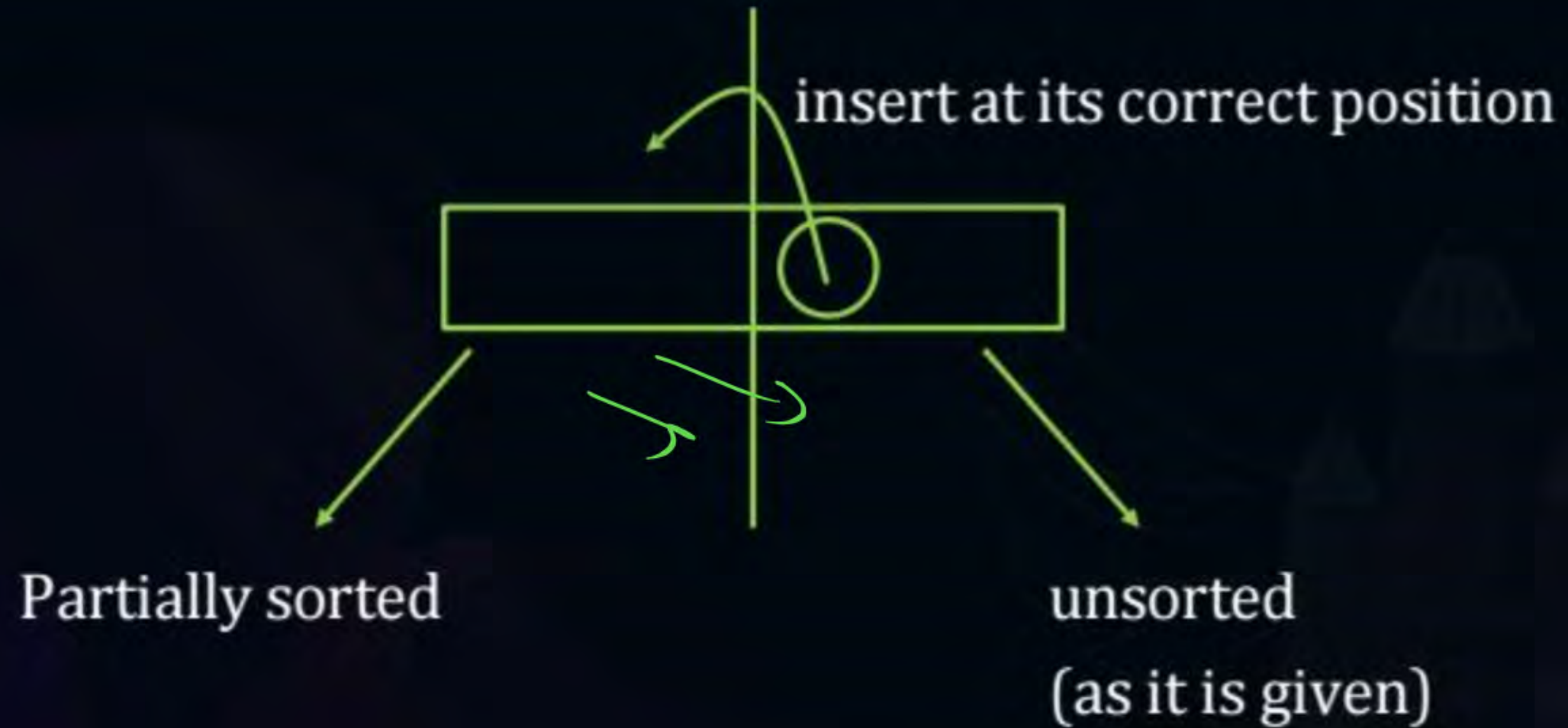


Telegram Link for Aditya Jain sir:
https://t.me/AdityaSir_PW



Topic : Sorting Algorithms

Logic:



A: [85 22 49 97 86 15]



Topic : Sorting Algorithms

Code Walkthrough:

85	22	49	97	86	15
----	----	----	----	----	----

Pass1: 85 | 22 49 97 86 15
Pass2: 22 85 | 49 97 86 15
Pass3: 22 49 85 | 97 86 15 → B.C of a pass
Pass4: 22 49 85 97 | 86 15
Pass5: 22 49 85 86 97 | 15 → W.C of a pass
(15 22 49 85 86 97) Sorted



Topic : Sorting Algorithms



Insertion Sort Code: 1 ^{based} ~~banded~~ indexing

Algo InsertionSort(A, n)

```
{  
    for (j = 2; j <= n; j++)  
    {  
        key = A[j]  
        i = j - 1  
        while (i > 0 and A[i] > key) {  
            A[i + 1] = A[i]  
            i --  
        }  
        A[i + 1] = key  
    }  
}
```

Dry Run



Topic : Sorting Algorithms

Dry-Run of Previous Insertion Sort Code:

i/p : A = ~~80~~ | 20 40 90 30 60

P1: j = 2;

key = A[j] = A[2] = 20

i = j - 1 = 2 - 1 = 1

while (i > 0 and A[i] > key)

{

| A[i + 1] = A[i] → *Shifting*

| i--

}

A[0 + 1] = 20

Handwritten arrow pointing to the result of the shift operation.

Handwritten formula: A[i+1] = A[i]



Topic : Sorting Algorithms



Dry-Run of Previous Insertion Sort Code:

A = 20 80 | 40 90 30 60

P2: j = 3; *80*

key = A[3] = 40

i = j - 1 = 3 - 1 = 2 *A[3] = 80*
i → 1

A[1 + 1] = 40



Topic : Sorting Algorithms

Dry-Run of Previous Insertion Sort Code:

A = 20 40 80 | 90 30 60

P3: j = 4;

key = A[4] = 90

i = j - 1 = 4 - 1 = 3

A[3 + 1] = key = 90

A[4] = 90

This is not a swap





Topic : Sorting Algorithms



Dry-Run of Previous Insertion Sort Code:

A = 20 40 80 90 | 30 60

P4: j = 5;

key = A[2] = key = 30

i = j - 1 = 5 - 1 = 4

i = 1

A[i+1] = 30



Topic : Sorting Algorithms

Dry-Run of Previous Insertion Sort Code:

A = 20 30 40 80 90 | 60

P5: j = 6;

key = A[4] = key = 60

i = j - 1 = 5 - 1 = 4

A[4 + 1] = 60

20 30 40 60 80 90 → Sorted





Topic : Sorting Algorithms



Time Complexity Analysis:

Worst Case:

i/p is sorted in opposite order

A = 80, 70, 65, 40, 10

Pass1 → 1 comp, 1 swap

Pass2 → 2 comp, 2 swap

Pass3 → 3 comp, 3 swap

.

.

Pass(n-1) → (n-1) comp, (n-1) swap

Total in all (n-1) passes

No. of comp

$$= 1 + 2 + 3 \dots (n-1) = \frac{n(n-1)}{2}$$

No. of swaps

$$= 1 + 2 + 3 \dots (n-1) = \frac{n(n-1)}{2}$$

Best Case : A is sorted

every pass : 1 Comp

Total $(n-1)$ Comp



Topic : Sorting Algorithms

Summary:

	No. of Comparisons	No. of Swaps (inversions)
Best case increasing order	$\{ (n - 1) +$	$0 \} \rightarrow \Omega(n)$
Worst Case Descending order	$\checkmark \left\{ \frac{n(n - 1)}{2} + \right.$	$\checkmark \left. \frac{n(n - 1)}{2} \right\} \rightarrow O(n^2)$

Space Complexity $\rightarrow O(1) \rightarrow$ Inplace





Topic : Sorting Algorithms



Important Points on Insertion Sort: V. Imp

1. Always takes $(n - 1)$ passes ✓
2. Time complexity
BC $\rightarrow \Omega(n)$
WC $\rightarrow \underline{O(n^2)}$
3. Space Complexity $\rightarrow \underline{O(1)} \rightarrow$ Inplace
4. Also stable
5. If the input list is pre-sorted, then it takes time of $O(n + d)$,
★ where, n = no. of elements
 d = no. of inversions



Topic : Sorting Algorithms



Note:

For 5^{th} pb

We use this while solving questions when the input sequence/order is not known and only no. of inversions are mentioned.

Radix Sort



Topic : Sorting Algorithms



Radix Sort (Non-Comparison based)

- No. of passes = no. of digits in the max element of given input
- Makes use of Buckets/Bins



Topic : Sorting Algorithms

Example: Base = 10 (decimal)

i/p: $A = [64, 723, 99, 83, 545, 65, 333, 7, 4, 124]$

Input : $\{L \rightarrow R\}$

o/p : Buckets $\{L \rightarrow R, \text{Bottom to Top}\}$

$\max(A) = \underline{\underline{723}}$

no. of digits (723) = 3 $\rightarrow$ means 3 passes

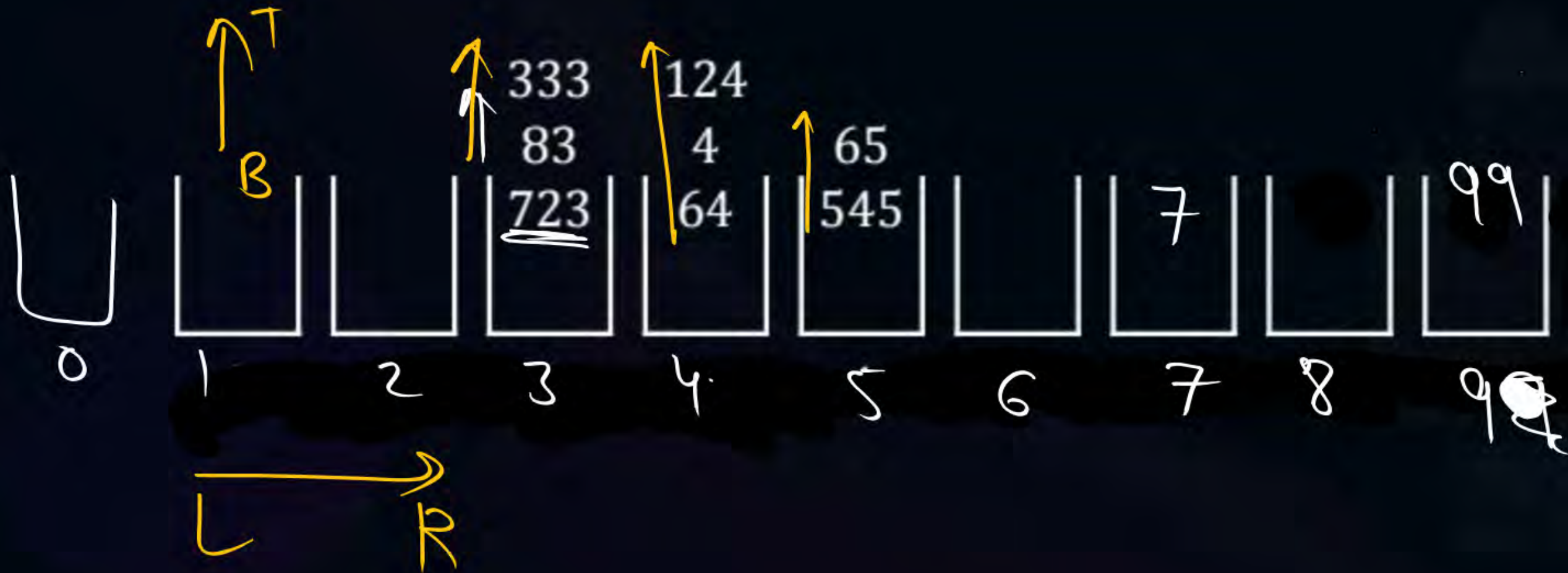


Topic : Sorting Algorithms

Pass1: Bucket as per unit place digit (Right most digit)

64, 723, 99, 83, 545, 65, 333, 7, 4, 124
↓4 ↓3 ↓9 ↓3 ↓5 ↓5 ↓3 ↓7 ↓4 ↓4

Pass1 o/p : [723, 83, 333, 64, 4, 124, 545, 65, 7, 99]





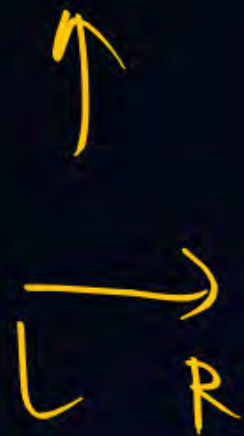
Topic : Sorting Algorithms

Pass2:

i/p : [723, 83, 333, 64, 4, 124, 545, 65, 7, 99]

↓ ↓ ↓ ↓ ↓ ↓ ↓ ↓ ↓ ↓
2 8 3 6 0 2 4 6 0 9

Pass2 o/p : [4, 7, 723, 124, 333, 545, 64, 65, 83, 99]



7		124				65			
4		723	333	545		64		83	99
0	1	2	3	4	5	6	7	8	9



Pass3:

o/p : [4, 7, 64, 65, 83, 99, 124, 333, 545, 723]



Final sorted o/p with total 0 comparisons.

9



Topic : Sorting Algorithms



Note:

TC :

Time Complexity: $O(d * (n+b)) \approx \underline{\underline{O(n * d)}}$

b = base of given input

d = no. of digits in max elements of given i/p array

n = no. of elements in given i/p array

Radix Sort → Not inplace
→ Stable

SC: $O(n+k)$

n → no. of elem

k → max elem in Arr

PYQs + Practice



Topic : Bubble Sort



#Q. If we use Radix Sort to sort n integers in the range $(n^{k/2}, n^k]$, for some $k > 0$ which is independent of n , the time taken would be:

$$(n^{k/2}, n^k]$$

30%

$$(n^{k/2}, n^k]$$

↓
max

A $\Theta(n)$ ✗

B $\Theta(kn)$ ✗

C $\Theta(n \log n)$ ✓

D $\Theta(n^2)$ ✗

1000

$$\left\lceil \log_{10}(10^3) \right\rceil + 1$$

$$n^K$$

$$d = \log(n^K) = \boxed{K * \log(n)}$$



Topic : Bubble Sort



#Q. Give the correct matching for the following pairs :

List-I

A. $O(\log n)$

B. $O(n)$

C. $O(n \log n)$

~~D. $O(n^2)$~~

Codes:

List-II

P. Selection

Q. Insertion sort

~~R. Binary search~~

~~S. Merge sort~~

A - R, B - Q, C - S, D - P

A

A - R B - P C - Q D - S ~~X~~

B

A - R B - ~~Q~~, C - ~~S~~, D - ~~P~~

C

A - P B - R C - S D - Q ~~X~~

D

A - P B - S C - R D - Q ~~X~~

B

18%



Topic : Bubble Sort



P4Q

#Q. If all permutations are equally likely, what is the expected number of inversions in a randomly chosen permutation of $1 \dots n$?

38.71.

A $\frac{n(n-1)}{2}$

☒ **B** $\frac{n(n-1)}{4}$

C $\frac{n(n+1)}{4}$

D $2n[\log_2 n]$

Solns:-

$\boxed{n}$

$$\text{No. of pairs} = {}^nC_2$$

$$= \frac{n!}{(n-2)! \times 2!} = \frac{n \times (n-1) \times \cancel{(n-2)!}}{\cancel{(n-2)!} \times 2}$$

$$= \boxed{\frac{n(n-1)}{2}}$$

$$\text{Prob}(\text{pair} = \text{inv}) = \frac{1}{2}$$

$$\text{Expected inv} = P(\text{inv}) \times \text{no. of inv}$$

$$= \frac{1}{2} \times \frac{n(n-1)}{2}$$

$$= \boxed{\frac{n(n-1)}{4}}$$





Topic : Bubble Sort



#Q. What would be the worst case time complexity of the insertion sort algorithm, if the inputs are restricted to permutation of $1 \dots n$ with at most n inversions?

A $\Theta(n^2)$ ~~X~~

B $\Theta(n \log n)$

C $\Theta(n^{1.5})$

D $\Theta(n)$

$$T_c = O(n + d)$$

$$d = \underline{O(n)}$$

$$T_c = O(n + n) = \underline{\underline{O(n)}}$$

32%



Topic : Bubble Sort



#Q. Assume that the algorithm considered here sort the input sequence in ascending order. If the input is already in ascending order, which of the following are TRUE?

59%

I. Quick sort runs in $\Theta(n^2)$ time.

II. Bubble sort runs in $\Theta(n^2)$ time. ~~X~~

III. Merge sort runs in $\Theta(n)$ time.

IV. Insertion sort runs in $\Theta(n)$ time. ~~✓~~

A

I and II only ~~X~~

B

I and III only ~~X~~

C

II and IV only ~~X~~

D

I and IV only ~~11/9~~

D



Topic : Bubble Sort



HQD

#Q. Consider the following array :

23	32	45	69	72	73	89	97
----	----	----	----	----	----	----	----

Which algorithm out of the following options uses the least number of comparisons (among the array elements) to sort the above array in ascending order ?

A

Quick sort using the last elements as pivot

B

Selection sort

C

Merge sort

D

Insertion sort



Topic : Bubble Sort



How

#Q. The usual $\Theta(n^2)$ implementation of Insertion Sort to sort an array uses linear search to identify the position where an element is to be inserted into the already sorted part of the array. If instead, we use binary search to identify the position, the worst case running time will.

- ☒ **A** Remains $\Theta(n^2)$
- ☐ **B** Become $\Theta(n (\log n)^2)$
- ☐ **C** Become $\Theta(n \log n)$
- ☐ **D** Become $\Theta(n)$



2 mins Summary



Topic

Topic

Topic

Topic

Topic

Insertion

Radix

PYQs



THANK - YOU